

SLIDE 1 — What is an Array?

Definition:

An **array** is a collection of multiple values of the **same data type** stored under **one variable name**.

Simple idea

One Name → Multiple Values → Index

Example:

```
int marks[5] = {85, 90, 78, 92, 88};
```

Explanation:

marks is an integer array that stores **5 marks**.

Each value has a position called an **index**.

```
marks[0] → 85  
marks[1] → 90  
marks[2] → 78  
marks[3] → 92  
marks[4] → 88
```

Output:

```
85  
90  
78  
92  
88
```

Analogy:

Student Marks → One Name → Multiple Boxes

```
marks  
↓  
[85] [90] [78] [92] [88]  
↓   ↓   ↓   ↓   ↓
```

[0] [1] [2] [3] [4]

 **Remember: Array = One variable, many values!**

SLIDE 2 — How Does an Array Work?

Structure

```
dataType arrayName[size];
```

Example

```
int numbers[5];
```

C creates space for **5 integer values**.

Explanation:

The array has **5 positions**, and each position is identified using an **index**.

Important:

Array indexing starts from **0**, not 1.

Index:	0	1	2	3	4
	↓	↓	↓	↓	↓
Array:	[10]	[20]	[30]	[40]	[50]

Accessing an element

```
printf("%d", numbers[2]);
```

If:

```
int numbers[5] = {10, 20, 30, 40, 50};
```

Then:

numbers[2] → 30

Remember

 **Index 0 → First element**

● Index 1 → Second element

● Index 2 → Third element

● Index starts from 0!

SLIDE 3 — What Can We Store in an Array?

Arrays can store multiple values of the same data type.

Integer Array

```
int ages[4] = {18, 20, 19, 21};
```

Stores: Integer values.

Float Array

```
float prices[3] = {10.5, 20.5, 30.5};
```

Stores: Decimal values.

Character Array

```
char vowels[5] = {'a', 'e', 'i', 'o', 'u'};
```

Stores: Character values.

Array Types

Data Type	Example
int	<code>int marks[5];</code>
float	<code>float price[3];</code>
char	<code>char vowels[5];</code>

```
double    double
          values[4];
```

Important

Array → Same data type

```
int numbers[3] = {10, 20, 30};
```

 Correct

```
int numbers[3] = {10, 20, 'A'};
```

 Not recommended for an integer array.

Remember

One array → Same type of data

SLIDE 4 — Arrays in Real Programs

Arrays help programs store and manage multiple values easily.

Student Marks

```
int marks[5] = {80, 75, 90, 85, 95};
```

Stores marks of multiple subjects/students.

Shopping

```
float prices[4] = {99.5, 150.0, 75.5, 200.0};
```

Stores prices of multiple products.

Temperature

```
float temperature[7];
```

Stores temperature for 7 days.

Games

```
int scores[5] = {100, 250, 180, 320, 400};
```

Stores scores of multiple players/rounds.



Data Processing

```
int numbers[5] = {10, 20, 30, 40, 50};
```

A program can calculate:

- Sum
- Average
- Maximum
- Minimum
- Search for a value



Basic Pattern

CREATE → STORE → ACCESS → PROCESS

Array is a basic **data-storage** building block.

SLIDE 5 — RECAP

C Can Store Multiple Values!

Flow

DECLARE



CREATE ARRAY



STORE VALUES



USE INDEX



ACCESS ELEMENT



5 Things to Remember

1. An array stores multiple values
2. All elements usually have the same data type
3. Array indexing starts from 0

4. Each element is accessed using its index

5. Array size tells how many elements it can hold

👁️ Final Example

```
int numbers[5] = {10, 20, 30, 40, 50};
```

```
printf("%d", numbers[2]);
```

Index 2 → 30

Output:

30

⚡ Memory Trick

Array → STORE MANY! 📦

Index → FIND ONE! 🔍

Index starts at 0! 0

QUESTIONS

1. What is the main purpose of an array in C?

- A) To repeat code
- B) To store multiple values of the same data type
- C) To print output
- D) To create a condition

💡 **Hint:** Think about how an array stores many values under one name.

📖 **Glossary:** Array → A collection of multiple values stored under one variable name.

✅ **Answer:** B) To store multiple values of the same data type

2. What is the first index of an array in C?

```
int numbers[5] = {10, 20, 30, 40, 50};
```

 **Hint:** C starts counting array positions from zero.

 **Glossary: Index** → The position number used to access an array element.

Answer:

0

The positions are:

0 → 10

1 → 20

2 → 30

3 → 40

4 → 50

3. What value is stored at `numbers[2]`?

`int numbers[5] = {10, 20, 30, 40, 50};`

 **Hint:** Remember that the first element is at index 0.

 **Glossary: Element** → An individual value stored inside an array.

 **Answer:** 30

4. Arrange the steps in the correct order:

A. Access an element

B. Declare an array

C. Store values

D. Use the index

 **Hint:** First create the array, then put values into it. To access a value, use its index.

 **Glossary: Declare** → To define a variable or array so C knows about it.

 **Answer:**

B → C → D → A

5. An array stores multiple values of the _____ data type.

💡 Hint: An integer array stores integers, and a float array stores floating-point values.

📖 Glossary: Data type → Defines what kind of value a variable can store.

✅ Answer: SAME**